Book-Guided, LLM-Detailed, Kernel-Verified: Formal Proof Construction from ZFC Axioms to Program Correctness

Yu Chenkan

2026

Abstract

Formal proofs are too large for humans to write, read, or verify by inspection. A two-line definition of addition on natural numbers expands to over 10,000 lines of formal proof from ZFC axioms. This 1000:1 ratio is not a tooling limitation — it is the inherent nature of formal verification.

We present a pipeline for formal proof construction that addresses this gap: a mathematical textbook provides proof strategy and development order (*book-guided*); a large language model translates informal reasoning into formal proof steps (*LLM-detailed*); and a minimal proof kernel guarantees correctness (*kernel-verified*).

We demonstrate this pipeline by constructing a verified proof chain from Zermelo set theory axioms to the correctness of a Turing machine that computes addition — spanning 10 abstraction levels, 199 theorems, and 38,247 lines of proof, all verified by a 334-line sequent calculus kernel. Three aspects of this result are notable: the proof uses only Zermelo set theory (Z), achieving tighter axiom bounds than typical formalizations; the proof chain connects arithmetic directly to computation within a single verified environment, demonstrating cross-abstraction reasoning; and the choice of ZFC over dependent type theory aligns the system with the native language of scientific textbooks. During formalization, the kernel caught a definition error in the guiding textbook that three years of human review had missed.

1 Introduction

The promise of formal verification is absolute correctness: if a proof checks, the theorem holds. The barrier is cost. Formal proofs in systems like Coq, Lean, Isabelle, and Metamath are orders of magnitude longer than their textbook counterparts, requiring extensive manual effort from trained experts.

Large language models (LLMs) offer a potential solution. They can generate large volumes of plausible formal text, and they can work at a scale of mechanical detail that humans cannot sustain. But LLMs are unreliable — they hallucinate, make logical errors, and produce subtly wrong proofs. Used alone, they cannot be trusted for formal verification.

We propose a pipeline that combines three components:

1. **Book-guided.** A mathematical textbook, written with formal modeling in mind, provides the proof strategy, definitions, and development order. The book is the human-readable specification of what should be proved and why.

2. **LLM-detailed.** A large language model translates the textbook's informal reasoning into formal proof steps. It fills in the mechanical detail, proposes intermediate lemmas when the proof gets stuck, and navigates the growing theorem library by semantic association.

3. **Kernel-verified.** A minimal proof kernel checks every proof step at construction time. If the kernel accepts a proof, it is correct regardless of whether the LLM made errors along the way. The kernel is small enough (334 lines) to be audited by a human.

The key insight is that these three components address different aspects of a structural problem. Formal proofs are humanly impossible to write (too long), humanly impossible to read (too detailed), and humanly impossible to verify by inspection (too complex). The textbook is the human interface. The LLM is the translator. The kernel is the guarantor.

We build on ZFC set theory rather than the dependent type theories used by Coq and Lean. ZFC is the native foundation of working mathematics and scientific modeling — sets, functions, and real numbers are first-class objects, not encodings. This makes the pipeline directly applicable to standard scientific textbooks.

We demonstrate the pipeline by constructing a verified proof chain from Zermelo set theory (Z) axioms to Turing machine correctness, spanning 10 abstraction levels (Section 4). The proof connects arithmetic ($\text{Plus}(a, b, c)$) directly to computation ($\text{TMReaches}(\delta, c_0, n, c_f)$) within a single verified environment — cross-abstraction reasoning that existing single-level tools cannot express. The proof achieves axiom minimality: only Z is needed, without Replacement, Regularity, or Choice. During formalization, the kernel caught a definition error in the guiding textbook that three years of human review had missed (Section 5).

2 The Scale of Formal Proof

The gap between informal mathematics and formal proof is not a small constant factor. To illustrate, consider the definition of addition on natural numbers in ZFC set theory.

Textbook definition. Two lines:

$$m + 0 = m$$
$$m + S(n) = S(m + n)$$

Formal proof. The proof that this function exists, is unique, commutes, associates, and computes concrete results such as $4+3=7$ occupies 10,720 lines of formal proof code. This covers a single arithmetic operation — addition — and does not include the prerequisite infrastructure (ordered pairs, functions, natural numbers, the recursion theorem) which adds approximately 27,500 more lines.

The ratio is roughly 1000:1. A page of textbook mathematics becomes a thousand lines of formal proof. This ratio is not specific to our system; it reflects the inherent gap between informal mathematical reasoning and the level of detail required for machine verification. Every implicit appeal to "it is easy to see," every omitted case analysis, every use of a previously established fact must be made explicit.

At this scale, three human limitations become absolute:

- **Humans cannot write these proofs.** The sheer volume of mechanical proof steps exceeds what a person can sustain. Writing 38,247 lines of correct proof code by hand, where each line must satisfy a proof kernel's rule checking, is not difficult — it is infeasible.
- **Humans cannot read these proofs.** The proof of a single arithmetic lemma may span hundreds of lines of tactic applications, variable instantiations, and cut compositions. No human can hold this in working memory or extract meaning from it by inspection.
- **Humans cannot verify these proofs.** Without a mechanical checker, there is no way to confirm that 38,247 lines of proof are correct. The proof exists beyond human verification capacity.

These are not limitations of proof engineering or tooling. They are structural consequences of what formal proof *is*. Any system that aspires to large-scale formal verification must reckon with this gap.

To make this concrete: the complete proof of TM addition correctness requires 134,089 individual kernel rule applications (counting each shared lemma once). These span 150 unique theorems across 10 abstraction levels. Each rule application is a sequent calculus step — an axiom introduction, a quantifier instantiation, a cut, a weakening — mechanically validated by the kernel at construction time. No human could perform or check 134,089 sequential logical steps by hand.

3 Architecture

3.1 Foundation: ZFC and Sequent Calculus

We choose Zermelo-Fraenkel set theory with Choice (ZFC) as the logical foundation, implemented via Gentzen's sequent calculus (LK).

The choice of ZFC over dependent type theories (the Calculus of Inductive Constructions used by Coq and Lean) is deliberate. Scientific and mathematical knowledge is overwhelmingly expressed in the language of sets, functions, and relations. ZFC is the native foundation of working mathematics. Encoding mathematical objects in dependent type theory introduces a translation layer — natural numbers become inductive types, functions become dependent products — that adds complexity without adding expressive power for the problems we target.

Sequent calculus provides a clean proof-theoretic framework with well-understood meta-properties (cut elimination, the subformula property). Proofs are explicit derivation trees, not tactic scripts, making verification straightforward.

3.2 The Proof Kernel

The proof kernel is a 334-line Python module. The complete trusted core — formula language (88 lines), ZFC axiom definitions (144 lines), derived connectives (119 lines), and the proof checker itself — totals 685 lines. The kernel implements:

- **Sequent representation.** A sequent $\Gamma \vdash \Delta$ consists of left (assumptions) and right (conclusions) formula lists with set semantics (no duplicates, checked via alpha-equivalence).
- **Eleven proof rules.** Logical rules (axiom, not-left, not-right, implies-left, implies-right, forall-left, forall-right, cut) and structural rules (weakening-left, weakening-right). Each rule is validated at proof construction time.
- **Lazy definition expansion.** Derived connectives (exists, and, or, iff, equality) and vocabulary predicates expand to primitive formulas on demand.
- **Alpha-equivalence.** Formula comparison respects bound variable renaming.

Every `Proof` object validates its rule application at construction. If the rule is misapplied — wrong principal formula, eigenvariable violation, context mismatch — construction raises an exception. There is no way to hold an invalid `Proof` object.

The final `qed()` function checks that a completed proof has exactly one formula on the right (the theorem) and only ZFC axiom instances on the left.

3.3 Vocabulary Expansion

Between the kernel and the theorems sits a vocabulary layer: named predicates that expand to first-order formulas. For example:

- Successor(s, n) expands to $\forall x.\, x \in s \leftrightarrow (x \in n \vee x = n)$
- Plus(m, n, p) expands via the recursion theorem's characterization of the addition function
- TMConfig(c, q, h, t) expands to an ordered-tuple encoding of Turing machine configurations

This layer is *outside* the trusted kernel. Vocabulary definitions can be wrong — but if they are, the proof will fail to go through. The kernel treats expanded formulas the same as any other formula.

3.4 LLM Integration

The LLM operates as an agent that edits proof source files. It reads theorem statements and existing proofs, proposes proof steps in Python, and runs the goal-checking harness. When the kernel rejects a step, the LLM reads the error message and adjusts.

The LLM's role is *search*: selecting which theorems to apply, in what order, with what instantiations. The kernel's role is *verification*: checking that each selected step is logically valid. The LLM can be wrong arbitrarily often; the kernel catches every error.

In practice, we observed that the LLM navigates the theorem library by *name semantics* — guessing that a theorem called `plus_val_unique` proves uniqueness of addition values, attempting to use it, and correcting the instantiation when the kernel rejects the first attempt. This pattern of semantic guessing followed by mechanical error correction proved effective as the theorem library grew. The vocabulary layer (Section 3.3) serves the same role: the LLM writes `Plus(m,n,p)` rather than its first-order expansion, using vocabulary names as semantic handles for reasoning.

4 Demonstration: ZFC to Program Correctness

We construct a complete formal proof of the following:

Theorem 1. *Given a Turing machine M with transition function δ, initial state q_0, and halting state q_H: for all natural numbers a and b, if M starts with a unary tape encoding of a and b (a ones, separator zero, b ones), then M halts with a tape encoding of $a + b$.*

The proof is deliberately restricted to Zermelo set theory (Z): Extensionality, Empty Set, Pairing, Union, Power Set, Separation, and Infinity. Replacement, Regularity, and Choice are not used.

4.1 Abstraction Levels

The proof chain crosses the following levels, each building on the previous:

1. **Logic.** Sequent calculus rules: modus ponens, double negation, quantifier instantiation, if-and-only-if introduction and elimination. (24 theorems)
2. **Set theory.** Empty set uniqueness, singleton and pair existence, union, power set, subset relations. (40 theorems)
3. **Ordered pairs.** Kuratowski encoding $\langle a, b\rangle = \{\{a\}, \{a, b\}\}$, injection (if $\langle a, b\rangle = \langle c, d\rangle$ then $a = c$ and $b = d$), existence. (included in sets)
4. **Functions.** Relations, single-valuedness, domain, range, application, function extensionality. (included in sets)
5. **Natural numbers.** Omega as the smallest inductive set, successor properties, transitive sets, omega induction. (9 theorems)
6. **Recursion.** The recursion theorem: for any function f and initial value a, there exists a unique function h with $\mathrm{dom}(h) = \omega$, $h(0) = a$, and $h(n^+) = f(h(n))$. (36 theorems)
7. **Arithmetic.** Addition as a recursive function on ω. Existence, uniqueness, commutativity, associativity, concrete computations. (44 theorems)
8. **TM definitions.** States, transitions, configurations, tapes as functions from positions to symbols.
9. **Tape operations.** Tape update existence, reading, function preservation across updates.
10. **Program correctness.** Five execution phases, each proved correct by omega induction or single-step verification, composed via a trace composition theorem (TMReachesCompose). (41 theorems)

Table 1 summarizes the proof chain.

Metric	Value
ZFC axioms	10 defined, 7 used (Z only)
Proof kernel / trusted core	334 / 685 lines
Total theorems	199
Total proof code	38,247 lines
Abstraction levels crossed	10
Development time	∼25 days
Per-theorem proof statistics (DAG)	
plus_comm	30 theorems, 11,593 steps
tm_add_correct	150 theorems, 134,089 steps

Table 1: Summary statistics of the ZFC-to-TM-correctness proof chain.

5 Book-Guided Formalization

The proof development followed a textbook written by the author over three years: a 219-page treatment of mathematical logic and software engineering covering propositional logic, first-order logic (including sequent calculus, cut elimination, and completeness), computability, first-order arithmetic (including Gödel's incompleteness theorems), ZFC set theory (including ordinals, cardinals, and the axiom of choice), and inner models (the constructible universe and relative consistency).

The engine's proof kernel implements the sequent calculus from Chapter 3 of this textbook. The theorem development follows Chapters 4.1 (ZFC axioms, basic set operations, ordered pairs, functions) and 4.2 (natural numbers, recursion). The Turing machine correctness proof extends beyond the textbook's content.

5.1 Example: The Recursion Theorem

The formalization of the recursion theorem (Theorem 4.2.14 of the textbook) illustrates the book-guided pipeline concretely. The textbook specifies both the theorem statement and the proof strategy: construct the recursive function h by taking the union of approximation functions, bounded within $\mathcal{P}(\mathcal{P}(\omega \cup \bigcup\bigcup f))$.

The LLM's first attempt used a simpler bounding set ($\mathcal{P}(\mathcal{P}(\omega))$) that required additional hypotheses not present in the textbook's formulation. When this approach proved unwieldy, the human architect directed the LLM to follow the book's specific bounding strategy. The LLM reimplemented the proof using $\mathcal{P}(\mathcal{P}(\omega \cup \bigcup\bigcup f))$, which eliminated the extra hypotheses and preserved the textbook's clean API.

During this process, the formalization uncovered a definition error in the textbook (described below). The full cycle — book provides strategy, LLM implements, kernel rejects, book's alternative approach adopted, LLM reimplements, kernel catches a book error — is recorded in the git history across 12 commits.

5.2 The Formalization Caught a Textbook Error

The textbook's definition of a recursive function is as follows:

Definition 1 (Textbook version)**.** *isRecursive*(h, a, f) *iff* *isFunction*$(h) \wedge h(0) = a \wedge \forall n \in \omega\ h(n^+) = f(h(n))$.

This definition contains no domain constraint on h. The textbook's proof derives $\mathrm{dom}(h) = \omega$ as a side result (page 142) but does not include it in the definition.

When the LLM attempted to formalize the uniqueness proof ($\exists! h$ isRecursive(h, a, f)), the kernel rejected it. The reason: without $\mathrm{dom}(h) = \omega$, two functions that agree on ω but differ on extraneous pairs both satisfy the definition. Uniqueness genuinely fails under the textbook's formulation.

The fix was to strengthen the definition:

Definition 2 (Corrected version)**.** *isRecursive*(h, a, f) *iff* *isFunction*$(h) \wedge$ *dom*$(h) = \omega \wedge h(0) = a \wedge \forall n \in \omega\ h(n^+) = f(h(n))$.

This error is characteristic of informal mathematics. The textbook proof works because the reader implicitly restricts attention to the "intended" function h. The kernel does not permit implicit restrictions. The definition must be self-contained.

The textbook was written by the author and reviewed over three years. The error was caught mechanically, in minutes, by a 334-line kernel during an LLM-driven formalization attempt.

6 Cross-Abstraction Depth

Existing formal verification tools typically operate at a single abstraction level. SMT solvers (Z3) handle decidable fragments. Interactive theorem provers (Coq, Lean) work within type theory. Model checkers (TLA+) reason about state machines. When a problem spans multiple levels — as most real systems do — these tools must be used in isolation, with the cross-level reasoning left to the human.

The proof of TM addition correctness is a concrete counterexample: a single proof chain, in a single system, verified by a single kernel, crossing from foundational set theory to program correctness. The tape is a set of ordered pairs. The head position is a natural number constructed via the recursion theorem. The correctness statement connects arithmetic ($\text{Plus}(a, b, c)$) to computation ($\text{TMReaches}(\delta, c_0, n, c_f)$).

This depth is harder to achieve than breadth. Breadth (more theorems at the same level) reuses established patterns. Depth (a new level on top of all previous levels) introduces new proof engineering challenges at every layer, and the proofs grow because they carry the entire pyramid beneath them.

This cross-abstraction depth manifests in the development process as well as in the final proof. When the LLM encountered missing lemmas during TM proof construction — for example, that addition values are unique, or that addition produces values no smaller than its inputs — it dropped to the arithmetic layer, proved the needed theorems, and resumed the TM proof.

Figure 1 shows the development trajectory. The first two weeks were spent on foundational infrastructure (logic, sets, ordered pairs, the recursion theorem) with slow growth. Once the recursion theorem was proved, arithmetic development accelerated sharply, and the TM correctness proof followed within days. The flat-then-steep curve illustrates how foundational work establishes patterns that accelerate higher-layer development.

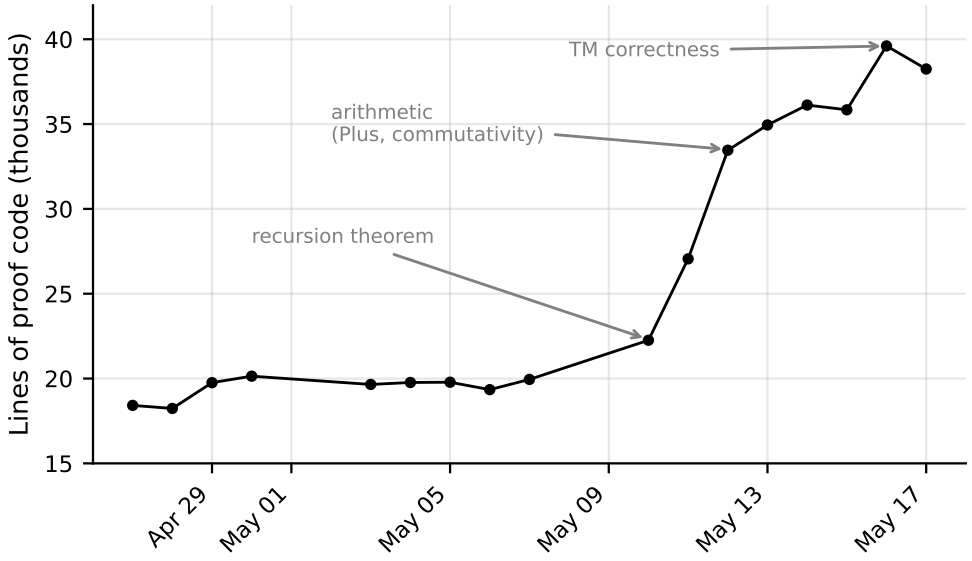

Figure 1: Total lines of proof code over the development period. The inflection point at the recursion theorem (May 10) shows how foundational depth enables rapid higher-level development.

7 Related Work

LLM-driven theorem proving. AlphaProof [Google DeepMind, 2024] combines a language model with AlphaZero-style reinforcement learning to solve competition mathematics in Lean. GPT-f [Polu and Sutskever, 2020] and follow-up work demonstrate LLM-based proof step generation in formal systems. These approaches use LLMs within existing proof assistants (Lean, Isabelle) and focus on search within a fixed logical framework. Our work differs in building the logical framework itself, using ZFC rather than dependent type theory, and targeting cross-abstraction reasoning rather than single-level proof search.

Interactive theorem provers. Lean/Mathlib [The mathlib Community, 2020], Coq [Bertot and Castéran, 2004], and Isabelle [Nipkow et al., 2002] are the dominant platforms for formalized mathematics. They are based on type theory (CIC or simple type theory) and rely on human-driven interactive proof construction. Mathlib contains over 100,000 theorems but has required a large community over many years. Our approach substitutes LLM assistance for most human effort, at the cost of a less mature library.

Set-theoretic formalization. Metamath's set.mm [Megill and Wheeler, 2019] is the largest ZFC-based formal library (∼40,000 theorems). Metamath uses a minimal substitution-based proof system; our use of Gentzen's sequent calculus (LK) provides a more natural proof-theoretic framework with well-studied meta-properties (cut elimination, the subformula property). Mizar [Bancerek et al., 2015] uses a set-theoretic foundation with a richer type system. Both rely entirely on human proof authors. Our contribution is demonstrating that LLM assistance can dramatically accelerate ZFC formalization while maintaining full kernel verification.

LLM as proof search heuristic. The use of neural networks as heuristics in formal reasoning has precedent in SAT solving [Selsam et al., 2019] and program synthesis. Our architecture makes this explicit: the LLM is an oracle that proposes proof steps, and the kernel is the verifier that accepts or rejects them. The LLM's reliability is irrelevant to the soundness of the final proof.

8 Discussion

Limitations. The current demonstration uses one textbook and one proof chain. Applying the pipeline to other textbooks and domains is the natural next step. Beyond the textbook's content (e.g., TM phase decomposition), a human architect provided proof strategy; extending textbook coverage would reduce this need.

The role of the textbook. Any scientific textbook that models its content mathematically is a potential input to this pipeline. The textbook provides what the LLM lacks: strategic direction, the right order of development, and conceptual organization. The LLM provides what manual formalization lacks: the ability to work at the scale of mechanical detail that formal proof requires. The "easy to see" steps in a textbook are exactly what LLMs are good at filling in — and the kernel catches the cases where "easy to see" is wrong.

Why ZFC. Beyond the proof-theoretic arguments of Section 3, ZFC is the native language of scientific modeling. A system targeting the formalization of knowledge across physics, engineering, and applied mathematics benefits from a foundation where sets, functions, and real numbers are first-class objects rather than encodings.

Axiom minimality as a strength. All 199 theorems use only Zermelo set theory (Z) — Replacement, Regularity, and Choice are not used. This is a deliberately stronger result than proving from full ZFC. The LLM's initial attempts used these stronger axioms freely; the human architect required proofs in Z alone, and the LLM found alternative constructions: successor injectivity via transitive sets (Lemma 4.2.4 of the textbook) rather than Regularity, and the recursion theorem's graph construction via power set bounding ($\mathcal{P}(\mathcal{P}(\omega \cup \bigcup\bigcup f))$) rather than Replacement. The kernel verified axiom minimality mechanically.

Depth as the hard direction. The demonstration in this paper prioritized depth (axioms to program correctness) over breadth (many theorems at one level). We believe this is the more important direction: depth demonstrates cross-abstraction reasoning, which is the capability that existing tools lack. Breadth can be added incrementally once the depth is established.

9 Conclusion

The gap between informal mathematics and formal proof is inherent, not a tooling limitation. The book-guided, LLM-detailed, kernel-verified pipeline addresses this gap by assigning each role to the component best suited for it: humans write textbooks, LLMs produce mechanical detail, and a small kernel guarantees correctness.

The formalized knowledge base — the theorems, their dependencies, the verified reasoning chains — is a cumulative asset. Each theorem makes future proofs easier. The library grows, and the LLM navigates it more effectively as it grows. Any scientific textbook that models its content mathematically is a potential input to this pipeline.

References

Grzegorz Bancerek et al. Mizar: State-of-the-art and beyond. *Intelligent Computer Mathematics*, 2015.

Yves Bertot and Pierre Castéran. *Interactive Theorem Proving and Program Development: Coq'Art: The Calculus of Inductive Constructions*. Springer, 2004.

Google DeepMind. Ai achieves silver-medal standard solving international mathematical olympiad problems. 2024. https://deepmind.google/discover/blog/ai-solves-imo-problems-at-silver-medal-level/.

Norman Megill and David A. Wheeler. Metamath: A computer language for mathematical proofs, 2019. http://us.metamath.org/.

Tobias Nipkow, Lawrence C. Paulson, and Markus Wenzel. *Isabelle/HOL: A Proof Assistant for Higher-Order Logic*. Springer, 2002.

Stanislas Polu and Ilya Sutskever. Generative language modeling for automated theorem proving. In *arXiv preprint arXiv:2009.03393*, 2020.

Daniel Selsam et al. Learning a sat solver from single-bit supervision. In *International Conference on Learning Representations*, 2019.

The mathlib Community. The lean mathematical library. *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*, 2020.